

GEPA: Reflective Prompt Evolution Can Outperform Reinforcement Learning

논문정리발표

숙명여자대학교 인공지능공학과 24학번 박채현
2026.02.21

목차 LIST

- 01 연구 배경 (Introduction)
- 02 GEPA 알고리즘 (1) - 핵심 원리
- 03 GEPA 알고리즘 (2) - 상세 프로세스
- 04 PUPA 유전트리
- 05 벤치마크 결과분석
- 06 결론 및 시사점 (Conclusion)

01 연구 배경 및 문제 제기(Introduction)

문제점

기존의 AI 학습 방식인 강화학습(RL, 예: GRPO)은
가중치를 직접 수정, 이 과정에서 수만 번의 연습(rollouts)이 필요, 비효율적

아이디어

숫자로 된 점수보다 자연어(글)가 훨씬 더 풍부한 정보를 담고
있으므로 AI가 스스로 자기 행동을 분석하고 글로써 반영하도
록 하면 더 빨리 배우지 않을까? 라는 사고에서 착안

목표

아주 적은 데이터로도 AI시스템을 최적화하는
GEPA(Genetic-Pareto) 알고리즘 제안

01 연구 배경 및 문제 제기(Introduction)

GEPA는 Genetic-Pareto Prompt Evolutionary Algorithm의 약자
쉽게 말해 "생물학적 진화 원리(Genetic)"와 "경제학적 최적화 개념(Pareto)"을 결합해서 AI프롬프트를 자동으로 고도화하는 도구

- 정체성: AI 모델의 가중치를 직접 건드리는 게 아니라, AI에게 주는 프롬프트를 진화시켜서 성능을 끌어올리는 프롬프트 최적화 시스템

파레토(Pareto): "어느 하나도 포기할 수 없는 최선"

02 GEPA 알고리즘의 핵심 메커니즘

자연어 성찰 및 변이 (Reflective Prompt Mutation)

- **실행 궤적(Execution Traces)수집**: AI가 문제를 풀때 남긴 사고체인, 도구호출내역, 평가 결과(에러메세지 등)을 모두 수집
- **성찰(Reflection)프로세스**: LLM이 이 흔적을 읽고, 여기서 왜 틀렸는지, 어떤 지침을 추가해야하는지 등 스스로 분석
- **고수준 규칙생성(Deriving High-level Rules)**: 그 결과로 나온 개선된 지침을 새로운 프롬프트 후보로 제안
- **변이(Mutation)**: 생성된 규칙을 기존 프롬프트에 결합하여 새로운 후보 프롬프트 생성, 이를 통해 모델이 경험하지 못한 새로운 문제에도 대처할 수 있는 일반화 능력을 얻음

02 GEPA 알고리즘의 핵심 메커니즘

파레토 기반 후보선택(Pareto-based Selection)

이 단계는 GEPA가 다른 최적화 도구와 차별화되는 가장 수학적이고 전략적인 부분

- **다양성 보존(Preserving Diversity):** 모든 문제에서 평균 점수가 높은 '무난한' 프롬프트 하나만 고르면, 특정 고난도 문제에서는 실패할 확률이 높음 이를 '지역 최적점(Local Optima)' 문제라고 함.
- **파레토 프런티어(Pareto Frontier) 구축:** GEPA는 각 학습 예제별로 성능을 측정. 예를 들어, '수학 문제에 강한 프롬프트'와 '추론 문제에 강한 프롬프트'가 있다면 둘 다 파레토 프런티어 세트에 포함시켜 보존.
- **선택 알고리즘:** 논문의 Algorithm 2(SelectCandidate)를 보면, 전체 후보군 중에서 다른 후보들에 의해 지배되지 않는 (Non-dominated), 즉 특정 영역에서 최고인 후보들만 골라 다음 세대의 '부모'로 삼음.

이렇게 하면 한 세대 안에서 다양한 해결 전략이 공존하게 되어, 시스템 전체의 견고함(Robustness)이 올라감.

02 GEPA 알고리즘의 핵심 메커니즘

시스템 인식 교차(System Aware Merge)

- 유전 알고리즘의 핵심인 '교차'를 프롬프트에 적용한 단계로, 서로 다른 장점을 가진 프롬프트들을 합치는 과정
- **구조적 결합**: 단순히 두 글을 이어 붙이는 게 아님 '교차 LLM'이 두 부모 프롬프트를 분석
- 부모 A: 시간 데이터를 처리하는 능력이 뛰어남
- 부모 B: 예외 상황에서 에러를 방지하는 지침이 강력함
- **상호보완적 통합**: 모델은 두 지침 중 서로 충돌하는 부분은 해결하고, 서로의 장점은 유지하면서 하나의 일관된 최종 진화형 프롬프트로 재구성
- 결과: 이 과정을 통해 생성된 프롬프트는 복잡한 제약 조건들을 모두 만족하도록 사람이 직접 설계하기 힘들 정도로 정교해짐

1. 초기화 및 후보 풀 (Candidate Pool P)

시작: 가장 먼저 기본이 되는 단순한 프롬프트들로 후보 풀(P)을 구성

구조: 왼쪽의 트리 구조처럼, 새로운 프롬프트는 이전 세대의 조상 프롬프트로부터 파생되어 계속 발전해 나감

2. 새로운 후보 제안 (Propose New Candidate)

기존 후보들 중 하나를 골라 더 똑똑하게 만드는 단계로, 두 가지 전략을 사용

성찰적 프롬프트 변이 (Reflective Prompt Mutation):

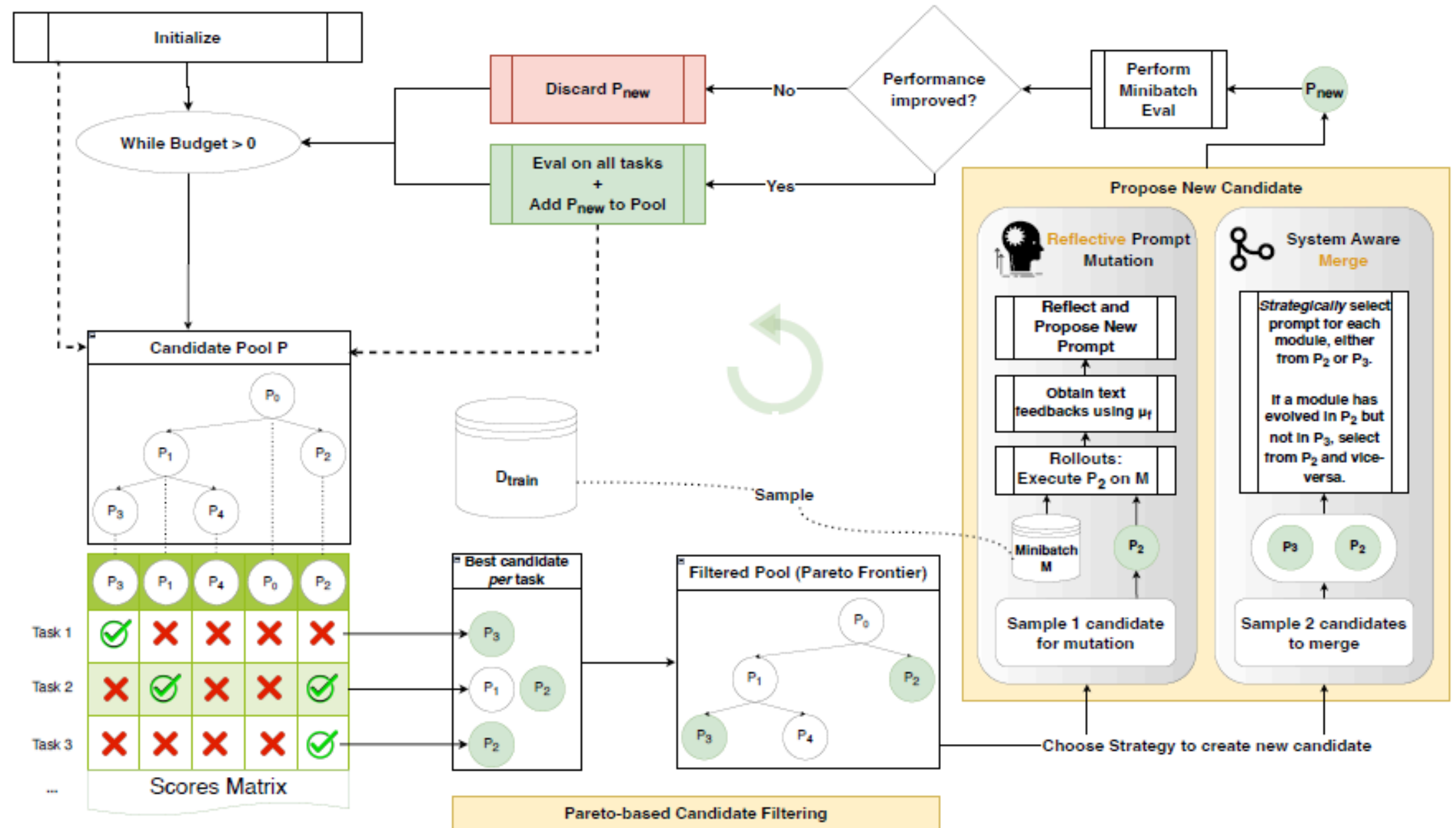
- 문제를 풀다가 틀린 부분(실행 궤적)을 보고 AI가 자연어로 피드백을 줌
- 이 피드백을 반영해 프롬프트 내용을 직접 수정(변이)

• 시스템 인지 병합 (System Aware Merge):

- 서로 다른 장점을 가진 프롬프트들을 전략적으로 결합하여 더 강력한 프롬프트를 만듦

3. 평가 및 필터링 (Evaluation & Filtering)

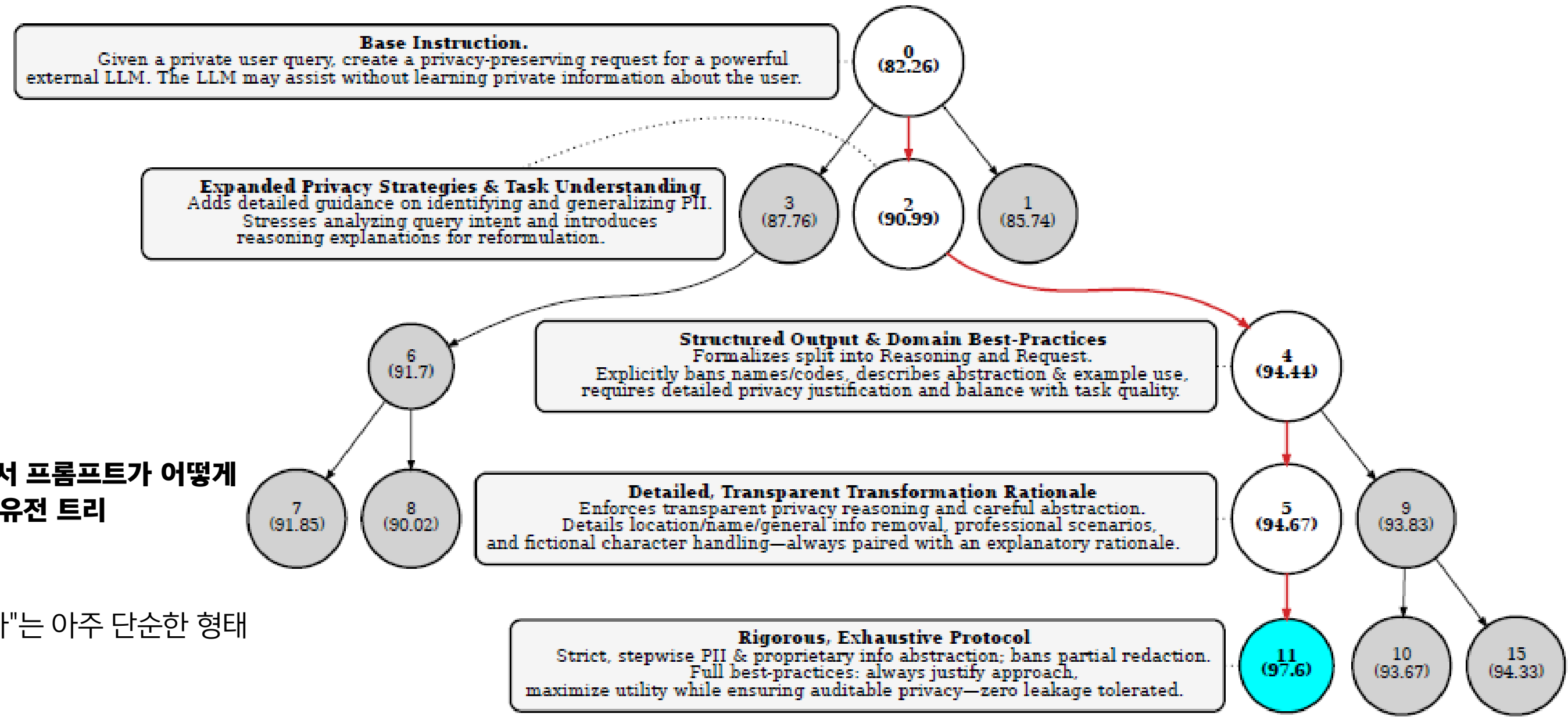
- 미니배치 평가: 새로 만든 프롬프트가 정말 좋은지 작은 데이터셋에서 먼저 테스트
- 성능 확인: 성능이 좋아졌다면 후보 풀에 추가하고, 그렇지 않으면 버림
- 파레토 기반 필터링 (Pareto-based Candidate Filtering):
 - GEPA의 핵심, 단순히 평균 점수가 높은 것만 남기는 게 아님.
 - 각 태스크별로 잘하는 프롬프트들이 다름, 다양한 유형의 문제를 골고루 잘 해결할 수 있도록 파레토 프런티어를 유지하며 최고의 후보군 선별



개인정보 보호 대리 권한 부여(PUPA) 작업에서 프롬프트가 어떻게 점진적으로 발전했는지를 보여주는 유전 트리

- 0단계: 기본 지시 (Base Instruction)
 - "개인정보를 보호하면서 요청을 처리하라"는 아주 단순한 형태 (성능 점수: 82.26).
- 중간 단계: 전략 확장 및 구조화
 - 노드 2: PII(개인 식별 정보) 식별 가이드를 추가하고 추론 설명을 도입(90.99).
 - 노드 4: 추론과 요청을 분리하고 도메인 모범 사례를 적용 (94.44).
- 최종 단계: 엄격한 프로토콜 (Rigorous Protocol)
 - 노드 11: 단계별 PII 추상화, 부분 편집 금지, 정보 유출 제로 보장 등 매우 정교한 프로토콜이 구축됨(97.6).

이처럼 GEPA는 매 단계마다 자연어 성찰을 통해 구체적인 뉘앙스를 추가하며 성능을 극대화



05 벤치마크 결과분석

가로축: Task

AI가 얼마나 똑똑한지 평가하는 테스트

- HotpotQA: 여러 개의 문서를 읽고 정보를 조합해서 답해야 하는 복합 추론시험.
- IFBench: "단어 3개 이하로 대답해", "특정 단어는 쓰지 마" 같은 까다로운 지시사항 을 얼마나 잘 지키는가
- Hover: 검색된 내용이 사실인지 아닌지 검증하는 능력 평가.
- PUPA: 개인정보를 보호하면서 사용자의 요청을 안전하게 처리하는지 평가

세로축: 최적화 방법 (Methods)

AI를 똑똑하게 만들기 위해 사용한 공부 방법

- Baseline: 아무런 최적화도 하지 않은 기본 상태
- MIPROv2: AI가 알아서 수천 개의 프롬프트 후보를 만들고 테스트해서 가장 좋은 걸 골라주는 자동 프롬프트 엔지니어
- GRPO (강화학습): AI가 맞으면 상을 주고 틀리면 벌을 주는 숫자 점수 기반 학습'
- GEPA: 이 논문에서 제안한 자연어 성찰 방식.
AI가 왜 틀렸는지 스스로 분석하며 발전함
- GEPA + Merge: GEPA 방식에 더해, 여러 개의 좋은 프롬프트를 하나로 합치는 전략'까지 사용한것

모델	최적화 방식	HotpotQA	IFBench	Hover	PUPA	Aggregate	향상도
Qwen3-8B	Baseline	42.33	36.90	35.33	80.82	48.85	—
	MIPROv2	55.33	36.22	47.33	81.55	55.11	+6.26
	GRPO	43.33	35.88	38.67	86.66	51.14	+2.29
	GEPA	62.33	38.61	52.33	91.85	61.28	+12.44
GPT-4.1 mini	Baseline	38.00	47.79	46.33	78.57	52.67	—
	GEPA	69.00	52.72	51.67	94.47	66.97	+14.29
	GEPA + Merge	65.67	55.95	56.67	96.46	68.69	+16.02

05 벤치마크 결과분석

구분	강화 학습 (예: GRPO)	GEPA
최적화 대상	모델의 내부 가중치 (Weights)	모델에 입력되는 프롬프트 (Prompts)
핵심 신호	숫자 점수 (Scalar Rewards)	자연어 성찰 (Textual Reflection) (구체적)
학습 메커니즘	경사 하강법 및 정책 최적화	유전 알고리즘 (변이, 선택, 병합)
데이터 효율성	낮음 (수만~수십만 번의 실행 필요)	매우 높음 (수백 번의 실행으로도 충분)
기본 기제	반복적인 보상을 통한 행동 강화	스스로 분석, 성찰을 통한 전략적 진화
프롬프트 관리	대개 고정된 프롬프트를 사용	파레토 프런티어를 통한 다양한 전략 보존

06 결론 및 시사점

1. 관련 연구 (Related Work)

프롬프트 최적화 (Prompt Optimization)

- 프롬프트 최적화는 거대 언어 모델(LLM)의 성능을 높이는 데 효과적임이 입증됨
- **환경으로부터 얻은 텍스트 피드백을 사용하고, 파레토 인지(Pareto-aware) 최적화를 통해 최적의 후보를 검색하며, 복합 AI 시스템의 각 서브 모듈에 맞게 프롬프트를 진화시킨다는 점에서 기존 연구와 다름**

진화형 LLM (Evolutionary LLMs)

- 알고리즘이 최적화된 프롬프트를 점진적으로 생성하는 방식(예: EvoPrompt)이 인기
- **기존의 방식들이 훈련 피드백을 잘 활용하지 못하거나 무작위 변이에 의존하는 것과 달리, GEPA는 도메인 특화된 피드백을 활용해 변이를 수행하여 훨씬 빠르게 수렴**

복합 AI 시스템 및 에이전트 최적화

- DSPy나 TextGrad 같은 기존 도구들은 최종적인 '전역 보상'에 의존
- **GEPA는 전역 보상을 신호로 쓰면서도 각 모듈에 대한 추가적인 텍스트 피드백을 포함. 또한, 전체 데이터를 하나로 보지않고 각 데이터 포인트에 맞는 프롬프트를 매칭하는 파레토 프론트 방식을 유지**

06 결론 및 시사점

2.한계점 및 향후 연구 (Limitations and Future Work)

- 데이터가 아주 많을 때는 프롬프트 수정보다 모델의 가중치를 직접 업데이트하는 방식이 더 나을 수 있음
- 현재 GEPA는 지침(Instruction)만 최적화하고 있는데, 앞으로는 예시(Exemplar)까지 최적화 대상에 포함할 수 있음
- 현재 GEPA의 실행 횟수 중 상당 부분이 후보 검증에 사용되는데, 이를 더 줄일 수 있는 동적 검증 전략이 필요
- 어떤 실행 기록이 성찰에 가장 도움이 되는지 찾아내는 '피드백 엔지니어링'도 유망한 연구 방향
- GEPA가 얻은 언어 기반 교훈을 강화 학습(RL)에 활용하는 등, 두 방식을 통합하는 시도를 계획 함

THANK YOU

감사합니다